Mô tả chi tiết mô hình AI

Nhận diện tiếng khóc trẻ sơ sinh trên ESP32 bằng Edge Impulse (TinyML) — dự án Smart Acoustic Node

1. Tóm tắt mô hình

Hạng mục	Nội dung
Project / impulse	Baby-cry-detector (ID 1117295); Time series data → Spectrogram → Classification (Keras)
Bản phát hành đang dùng	Thư viện Arduino v1.0.3; Edge Impulse Studio 1.95.13; deploy version 3; mã mô hình sinh ngày 20/09/2026
Bài toán	Phân loại âm thanh nhị phân trên mỗi cửa sổ 1 giây: crying (tiếng khóc) hoặc noise (tạp âm)
Đầu vào	16.000 mẫu, 16 kHz, mono, 16-bit (1 giây), đọc từ micro INMP441 qua I2S
Đặc trưng	Spectrogram 99 khung × 65 tần số = 6.435 giá trị, chuẩn hóa về khoảng 0–1
Mạng nơ-ron	CNN 1 chiều: Conv1D (8 filter, kernel 3) → MaxPool (2) → Dense (256) → Flatten → Dense (2) → Softmax
Quy mô	29.474 tham số; 282.440 phép nhân-cộng mỗi lần suy luận; 30.272 byte hằng số mô hình (trọng số INT8 + bias)
Lượng tử hóa / engine	INT8 toàn phần; EON Compiler (mã C++ biên dịch sẵn, chỉ 5 loại toán tử)
Bộ nhớ	Vùng nhớ tạm (tensor arena) 28.864 byte; Studio ước tính RAM đỉnh 49,1 KB, flash 72,2 KB
Thời gian trên ESP32	DSP 45–49 ms + mạng 34 ms = 79–83 ms mỗi cửa sổ (Studio ước tính suy luận 25 ms)
Dữ liệu	864 mẫu (674 huấn luyện, 190 kiểm thử), tổng 17 phút 7 giây; crying 5 phút 25 giây, noise 11 phút 42 giây
Độ chính xác (Studio)	Tập kiểm định 92,5%; tập kiểm thử 93,89% (bản INT8, chưa bật tăng cường dữ liệu)
Hậu xử lý trên thiết bị	Hysteresis: vào trạng thái khóc khi điểm crying ≥ 0,80, thoát khi < 0,50
Phần cứng	ESP32-WROOM + INMP441 (WS = GPIO15, SCK = GPIO14, SD = GPIO32, kênh trái)

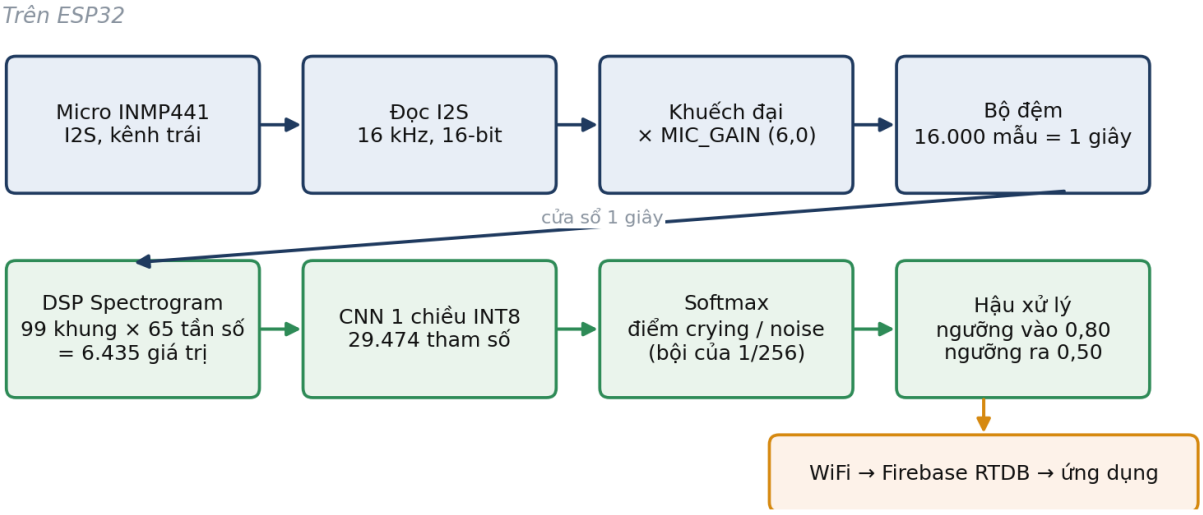
2. Bài toán và các ràng buộc

Mục tiêu của mô hình là phát hiện tiếng khóc của trẻ ngay trên vi điều khiển, để thiết bị chỉ gửi kết quả (nhãn, độ tin cậy, thời gian) lên đám mây mà không gửi âm thanh. Mô hình phải chạy trong các ràng buộc sau:

- Tài nguyên: RAM ESP32 là 327.680 byte (320 KB), không có GPU; firmware đầy đủ (mô hình, WiFi, Firebase) đã dùng 85.440 byte RAM tĩnh (26%) và 1.323.272 byte bộ nhớ chương trình.
- Thời gian: một lần phân loại mỗi cửa sổ 1 giây; xử lý phải nhỏ hơn nhiều so với 1 giây (thực tế 79–83 ms).
- Độ tin cậy: ưu tiên hạn chế cảnh báo giả, vì phụ huynh sẽ mất tin tưởng nếu hệ thống báo sai liên tục, nhưng vẫn phải bắt được tiếng khóc trong môi trường có tạp âm.

3. Luồng xử lý

Âm thanh đi qua tám bước trên thiết bị trước khi trở thành một cảnh báo (Hình 1). Bốn bước đầu (micro, đọc I2S, khuếch đại, bộ đệm) và bước hậu xử lý do firmware đảm nhiệm; ba bước giữa (Spectrogram, CNN, Softmax) nằm trong thư viện Edge Impulse và được gọi bằng một hàm duy nhất là `run_classifier()`.



Hình 1. Luồng xử lý âm thanh từ micro đến cảnh báo.

4. Dữ liệu huấn luyện

4.1. Nhóm em lấy dữ liệu từ đâu

Bộ dữ liệu của nhóm em có hai lớp là crying (tiếng khóc em bé) và noise (tiếng ồn). Dữ liệu lấy từ hai nguồn: khoảng 60% do nhóm tự thu bằng micro INMP441 gắn trên ESP32 (bằng chương trình thu âm nhóm tự viết), khoảng 40% còn lại tải từ Freesound và AudioSet rồi đổi về cùng định dạng mono, 16-bit. Tỉ lệ 60/40 này chưa được đối chiếu lại với số mẫu trên Studio.

Lớp noise gồm các loại tiếng ồn: tiếng ồn trong quán cà phê, tiếng ồn ngoài xóm, tiếng tivi và tiếng quạt.

4.2. Bộ dữ liệu trên Edge Impulse

Các số liệu dưới đây lấy từ tab Dataset và Create impulse của Edge Impulse Studio.

Hạng mục	Giá trị
Tổng dữ liệu	17 phút 7 giây (1.027 giây)
Số mẫu	864 mẫu: 674 mẫu huấn luyện và 190 mẫu kiểm thử (tỉ lệ 78% / 22%)
Lớp crying	5 phút 25 giây (325 giây), khoảng 32%
Lớp noise	11 phút 42 giây (702 giây), khoảng 68%. Con số này tính bằng tổng trừ thời lượng crying, vì bộ dữ liệu chỉ có hai lớp
Độ dài trung bình	khoảng 1,2 giây mỗi mẫu (1.027 giây chia 864 mẫu)
Cắt cửa sổ khi huấn luyện	Cửa sổ 1.000 ms, bước (stride) 500 ms, tần số 16.000 Hz, bật zero-pad, dùng 100% dữ liệu (Train on data subset)

Với bước 500 ms, một mẫu dài 10 giây được cắt thành 19 cửa sổ chồng lấn, còn một mẫu dài 1 giây chỉ cho 1 cửa sổ.

4.3. Một số điều nhóm em nhận thấy

- Có hai nhóm mẫu với độ dài khác nhau. Trong danh sách mẫu có các mẫu dài 10 giây (thêm vào ngày 27/09/2026) và các mẫu dài 1 giây (ngày thêm 05/04/2022). Vì độ dài trung bình chỉ khoảng 1,2 giây nên phần lớn mẫu là mẫu ngắn khoảng 1 giây.
- Lớp noise nhiều hơn lớp crying. Tiếng khóc chỉ chiếm khoảng 32% thời lượng, noise nhiều gấp khoảng 2,2 lần. Sự chênh lệch này có thể là một lý do khiến mô hình bỏ sót tiếng khóc nhiều hơn báo nhầm (mục 8), nhưng nhóm em chưa kiểm chứng riêng.
- Nhãn chung cho cả file dài. Edge Impulse cắt file dài thành nhiều cửa sổ chồng lấn nhưng gán cùng một nhãn của file gốc, nên đoạn yên tĩnh giữa các tràng khóc bị gán nhầm là crying và làm nhiễu quá trình học. Cách xử lý là cắt file thành các đoạn ngắn có nội dung đồng nhất.

- Dữ liệu từng quá hẹp. Một phiên bản mô hình trước từng có số liệu kiểm thử rất tốt nhưng nhận nhầm im lặng thành tiếng khóc khi chạy trên phần cứng thật. Sau khi thêm nhiều loại tạp âm, lớp noise đạt khoảng 100% trên tập kiểm thử của phiên bản đó. Bài học là số liệu trên Studio chưa đủ, phải thử bằng phần cứng thật với âm thanh mới.

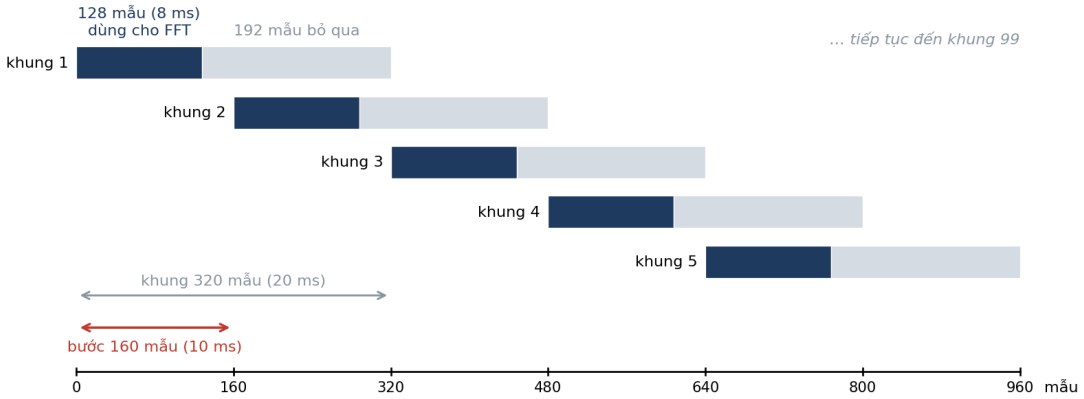
5. Tiền xử lý tín hiệu: khối Spectrogram (DSP)

Mạng nơ-ron không nhận sóng âm thô mà nhận một ảnh phổ (thời gian × tần số) do khối DSP tính ngay trên ESP32. Các tham số của khối này lấy từ `model_variables.h`:

Tham số	Giá trị	Ý nghĩa
Độ dài khung (frame length)	0,02 s = 320 mẫu	Mỗi khung dài 20 ms
Bước khung (frame stride)	0,01 s = 160 mẫu	Hai khung liên tiếp cách nhau 10 ms (chồng lấn 50%)
Độ dài FFT	128	Cho 65 hệ số tần số (0 đến 8 kHz, cách nhau 125 Hz)
Ngưỡng nhiễu nền	−52 dB	Thành phần yếu hơn mức này được đưa về 0
Phiên bản thuật toán	3	Chuẩn hóa về khoảng 0–1 và cắt ở 1
Số khung / đầu ra	99 khung × 65 = 6.435	Khớp với kích thước đầu vào của mạng (6.435)

5.1. Các bước tính (đối chiếu với mã nguồn)

1. Chuẩn hóa biên độ. Firmware đưa 16.000 mẫu int16 vào dưới dạng số thực (giữ nguyên giá trị); khối DSP chia cho 32.768 để đưa về khoảng −1…1.
2. Chia khung. Cắt thành các khung 320 mẫu, bước 160 mẫu, không đệm thêm số 0: số khung = (16.000 − 160) / 160 = 99.
3. FFT 128 điểm. Với mỗi khung, chỉ 128 mẫu đầu được đưa vào FFT (khung 320 mẫu bị cắt theo độ dài FFT), không áp hàm cửa sổ như Hamming hay Hann. Kết quả là 65 hệ số tần số, tần số bin thứ k là 125 × k Hz.
4. Công suất. P[k] = |X[k]|² / 128.
5. Chuẩn hóa dB. f[k] = giới hạn trong [0, 1] của $(10 \cdot \log_{10}(P[k]) + 52) / 64$. Công suất dưới −52 dB cho giá trị 0; từ +12 dB trở lên cho giá trị 1 (dải động 64 dB).
6. Xếp thành đặc trưng. Ma trận 99 × 65 được trải theo từng khung thành vector 6.435 giá trị đưa vào mạng.



Hình 2. Chia khung và vùng được đưa vào FFT (5 khung đầu của cửa sổ 16.000 mẫu).

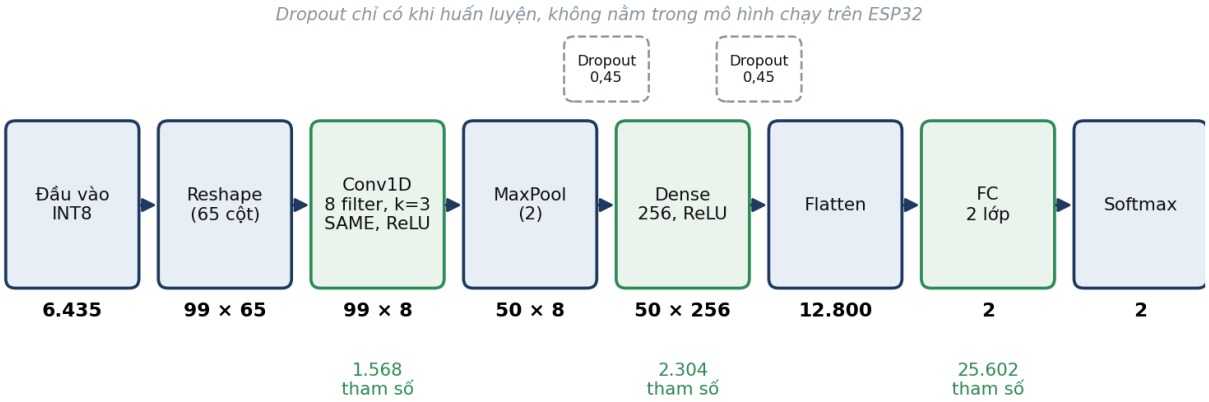
5.2. Những điều rút ra từ cách tính này

- Phủ khoảng 80% tín hiệu. Mỗi khung chỉ phân tích 128 mẫu (8 ms) và các khung cách nhau 160 mẫu, nên 128 trên mỗi 160 mẫu được dùng; 32 mẫu (2 ms) giữa các đoạn bị bỏ qua. Đây là đặc điểm của cấu hình FFT ngắn hơn khung.
- Độ phân giải: 125 Hz theo tần số và 10 ms theo thời gian.

- Hệ số khuếch đại ảnh hưởng trực tiếp tới đầu vào của mạng. Nhân biên độ với g làm mọi ô của ảnh phổ tăng thêm $20 \cdot \log_{10}(g)$ dB, tức khoảng 15,6 dB với ×6 và 18,1 dB với ×8. Trên thang 64 dB, đó là dịch chuyển khoảng 0,24 (×6) và 0,28 (×8) trong giá trị 0–1 trước khi bị cắt ở 1. Vì vậy MIC_GAIN phải được chọn sao cho mức phổ của INMP441 gần với mức của dữ liệu huấn luyện.

6. Kiến trúc mạng nơ-ron

Cấu hình lớp trên Edge Impulse Studio và các toán tử trong mã đã biên dịch khớp nhau. Mạng là CNN 1 chiều chạy dọc trục thời gian: sau lớp Reshape, mỗi hàng là một khung 10 ms và 65 cột là 65 tần số, nên mỗi filter của lớp Conv1D nhìn 3 khung liên tiếp trên toàn bộ 65 tần số.



Tổng: 29.474 tham số, 282.440 phép nhân-cộng mỗi lần suy luận. Lớp FC cuối chiếm 25.602 tham số (khoảng 87%).

Hình 3. Kiến trúc mạng và kích thước tensor ở từng lớp.

Bảng 1. Các lớp của mạng, kích thước tensor và số tham số (tính từ mã đã biên dịch).

#	Lớp (Studio)	Toán tử trong mã biên dịch	Vào → ra	Tham số	Phép nhân-cộng
1	Input	–	6.435 (INT8)	–	–
2	Reshape (65 cột)	RESHAPE	6.435 → 99 × 65	0	0
3	1D conv / pool: conv 8 filter, kernel 3, SAME, ReLU	CONV_2D	99 × 65 → 99 × 8	1.568	154.440
4	1D conv / pool: max pool 2	MAX_POOL_2D	99 × 8 → 50 × 8	0	–
5	Dropout 0,45	không có khi suy luận	–	0	–
6	Dense 256 nơ-ron, ReLU	CONV_2D (1 × 1)	50 × 8 → 50 × 256	2.304	102.400
7	Dropout 0,45	không có khi suy luận	–	0	–
8	Flatten	RESHAPE	50 × 256 → 12.800	0	0
9	Output (2 lớp)	FULLY_CONNECTED	12.800 → 2	25.602	25.600
10	Softmax	SOFTMAX	2 → 2	0	–
	Tổng			29.474	282.440

6.1. Tham số huấn luyện

Tham số	Giá trị
Số chu kỳ huấn luyện (training cycles)	100
Tốc độ học (learning rate)	0,008
Bộ tối ưu học được (learned optimizer)	Không dùng
Bộ xử lý huấn luyện	CPU
Tăng cường dữ liệu (data augmentation)	Không dùng ở mô hình gốc (xem mục 8.2)
Dropout	0,45 sau lớp pool và sau lớp Dense

6.2. Nhận xét về kiến trúc

- Mô hình rất nhỏ. Trọng số INT8 chiếm 29.208 byte và bias INT32 chiếm 1.064 byte (tổng 30.272 byte), vừa với ESP32 với nhiều dư địa.
- Phần lớn tham số nằm ở lớp cuối. Lớp Output nhận 12.800 giá trị (50 bước × 256) và có 25.602 tham số, khoảng 87% toàn mạng. Điều này làm tăng nguy cơ quá khớp, và hai lớp Dropout 0,45 được dùng để giảm nguy cơ đó.
- Mạng nhạy với vị trí trong cửa sổ. Không có lớp gộp toàn cục theo thời gian; lớp cuối dùng trọng số riêng cho từng bước thời gian. Vì vậy cùng một tiếng khóc xuất hiện ở đầu hay cuối cửa sổ 1 giây có thể cho điểm khác nhau. Đây là suy luận từ cấu trúc mạng, có thể góp phần vào việc điểm số dao động mạnh giữa các cửa sổ ở mục 9, nhưng chưa được kiểm chứng riêng.

7. Lượng tử hóa và vận hành trên thiết bị

Hạng mục	Giá trị	Ghi chú
Kiểu dữ liệu đầu vào / đầu ra	INT8 / INT8	Lượng tử hóa toàn phần
Lượng tử hóa đầu vào	scale 1/255, zero point −128	Đặc trưng 0–1 được ánh xạ lên −128...127
Lượng tử hóa đầu ra	scale 1/256, zero point −128	Điểm = (q + 128) / 256
Engine	EON Compiler (EI_CLASSIFIER_COMPILED = 1)	Mạng biên dịch thành mã C++, không cần bộ thông dịch
Toán tử sử dụng	RESHAPE, CONV_2D, MAX_POOL_2D, FULLY_CONNECTED, SOFTMAX	5 loại
Vùng nhớ tạm (tensor arena)	28.864 byte	Bản mặc định; 29.888 byte chỉ áp dụng cho cấu hình Himax
Ngưỡng tin cậy mặc định của thư viện	0,6	EI_CLASSIFIER_THRESHOLD; firmware dùng ngưỡng riêng 0,80 / 0,50
Cửa sổ / lát cắt	16.000 mẫu / 4 lát (4.000 mẫu = 250 ms)	Dùng khi chạy suy luận liên tục (run_classifier_continuous)

7.1. Cách đọc điểm số

Đầu ra Softmax được lượng tử hóa INT8 nên điểm p = (q + 128) / 256 với q từ −128 đến 127. Do đó điểm chỉ nhận các giá trị là bội của 1/256: nhỏ nhất 0,000 và lớn nhất 0,996 (255/256), không bao giờ bằng 1,000. Đối chiếu 40 giá trị điểm khác nhau trong các nhật ký Serial của nhóm, tất cả đều là bội của 1/256 (sai lệch làm tròn không quá 0,0006), xác nhận nhật ký đến từ đúng mô hình INT8 này.

Thứ tự nhãn trong thư viện là {crying, noise}; firmware tìm điểm crying bằng cách so sánh tên nhãn chứ không dựa vào chỉ số.

8. Kết quả đánh giá trên Edge Impulse

8.1. Mô hình gốc (không tăng cường dữ liệu)

Bảng 2. Chỉ số của mô hình gốc (bản INT8).

Chỉ số	Tập kiểm định	Tập kiểm thử
Accuracy	92,5%	93,89%
Loss	0,25	–
Area under ROC Curve	0,92	0,95
Weighted Precision / Recall / F1	0,92 / 0,92 / 0,92	0,95 / 0,95 / 0,95

Bảng 3. Ma trận nhầm lẫn (đơn vị: % theo hàng).

Tập	Thực tế \ Dự đoán	crying	noise	uncertain	F1
Kiểm định	crying	90,8%	9,2%	–	0,92
Kiểm định	noise	6,1%	93,9%	–	0,93
Kiểm thử	crying	93,5%	6,5%	0%	0,94
Kiểm thử	noise	2,9%	94,1%	2,9%	0,95

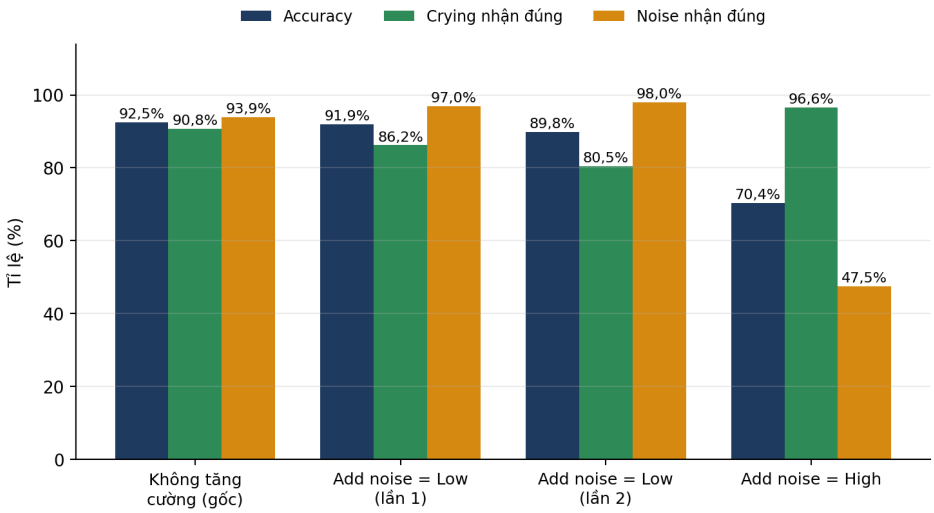
Độ chính xác trên tập kiểm thử (93,89%) không thấp hơn tập kiểm định (92,5%) và AUC đạt 0,95, nên chưa thấy dấu hiệu quá khớp đáng kể. Ở cả hai tập, tỉ lệ bỏ sót tiếng khóc (6,5–9,2%) cao hơn tỉ lệ báo nhầm (2,9–6,1%). Hai tập này lấy từ cùng nguồn dữ liệu, nên chúng phản ánh chất lượng trên dữ liệu quen thuộc; kết quả trên phần cứng (mục 9) mới là thước đo cuối cùng.

8.2. Thí nghiệm tăng cường dữ liệu (Add noise)

Nhóm em bật Data augmentation và đổi mức Add noise; các tùy chọn khác giữ cố định (Mask time bands: Low, Mask frequency bands: None, Warp time axis: tắt), kiến trúc và tham số huấn luyện giữ nguyên. Mức Low được huấn luyện hai lần.

Bảng 4. So sánh các cấu hình tăng cường dữ liệu trên tập kiểm định.

Cấu hình	Accuracy	Loss	Crying nhận đúng	Noise báo nhầm thành crying	Noise nhận đúng	AUC
Không tăng cường (gốc)	92,5%	0,25	90,8%	6,1%	93,9%	0,92
Add noise = Low, lần 1	91,9%	0,38	86,2%	3,0%	97,0%	0,92
Add noise = Low, lần 2	89,8%	0,24	80,5%	2,0%	98,0%	0,89
Add noise = High	70,4%	0,47	96,6%	52,5%	47,5%	0,72



Hình 4. Accuracy và tỉ lệ nhận đúng của hai lớp theo từng lần huấn luyện.

- Mức High không dùng được: accuracy 70,4%, AUC 0,72 và hơn một nửa mẫu noise (52,5%) bị đoán thành crying, tức là hệ thống sẽ báo động giả liên tục.

- Cùng cấu hình nhưng hai lần chạy cho kết quả khác nhau. Mức Low cho accuracy 91,9% ở lần 1 và 89,8% ở lần 2; tỉ lệ crying nhận đúng là 86,2% và 80,5%. Nếu dữ liệu không đổi giữa hai lần thì sự khác nhau này đến từ yếu tố ngẫu nhiên khi huấn luyện (khởi tạo mạng, nhiễu cộng ngẫu nhiên). Vì vậy chênh lệch nhỏ như 0,6 điểm giữa mô hình gốc và mức Low (lần 1) chưa có ý nghĩa, và mô hình gốc cũng mới chỉ chạy một lần.
- Hướng thay đổi thì giống nhau ở cả hai lần Low. So với mô hình gốc, noise nhận đúng tăng (97,0% và 98,0% so với 93,9%) còn crying nhận đúng giảm (86,2% và 80,5% so với 90,8%). Nói cách khác mức Low làm mô hình thận trọng hơn: ít báo giả hơn nhưng bỏ sót tiếng khóc nhiều hơn. Mới có hai lần chạy nên đây chỉ là xu hướng.
- Chưa có cơ sở chọn mức Low thay cho mô hình gốc. Về nguyên tắc, tăng cường dữ liệu chỉ tác động lên dữ liệu huấn luyện còn tập kiểm định không được cộng thêm nhiễu, nên lợi ích chịu nhiễu chỉ kiểm chứng được bằng thử nghiệm trên phần cứng.

9. Chạy trên ESP32 và kết quả thực nghiệm

9.1. Thu âm và gọi mô hình

Firmware cấu hình I2S ở chế độ master nhận, 16 kHz, 16-bit, kênh trái (chân L/R nối GND), DMA 8 bộ đệm × 512 mẫu. Mỗi vòng lặp thu đủ 16.000 mẫu (32.000 byte), nhân hệ số khuếch đại `MIC_GAIN` có chống tràn số, rồi đưa vào mô hình bằng hàm `run_classifier()` của thư viện Edge Impulse. Kết quả trả về là điểm tin cậy cho hai lớp crying và noise.

9.2. Hậu xử lý (hysteresis)

Điểm crying của từng cửa sổ dao động mạnh nên firmware dùng hai ngưỡng thay vì một: vào trạng thái khóc khi điểm ≥ 0,80 và chỉ thoát khi điểm < 0,50. Ở vùng 0,50–0,80 trạng thái được giữ nguyên. Chỉ khi chuyển từ yên bình sang khóc mới ghi một bản ghi lịch sử lên Firebase, nên mỗi đợt khóc chỉ có một bản ghi.



Hình 5. Máy trạng thái của bộ lọc hysteresis.

Lưu ý: các lần thử phần cứng dưới đây dùng bản firmware báo ngay khi một cửa sổ có điểm crying ≥ 0,80 (chưa có hysteresis).

9.3. Hệ số khuếch đại và biên độ tín hiệu

Bảng 5. Biên độ tín hiệu theo hệ số khuếch đại.

Hệ số	Kịch bản	Peak (nhỏ nhất–lớn nhất)	Peak lớn nhất / thang 16-bit	rms
×1	Khóc, nhiều tạp âm (gồm đoạn cuối không có âm thanh)	152–2.752	8,4%	37,6–438,6
×6	Khóc, một người nói chuyện bên cạnh	2.154–13.734	41,9%	552,4–3.255,0
×8	Khóc, nhiều tạp âm bên ngoài	2.856–28.000	85,5%	659,2–5.696,3
×8	Chỉ có tạp âm, không phát tiếng khóc	7.224–22.408	68,4%	1.834,3–3.172,7

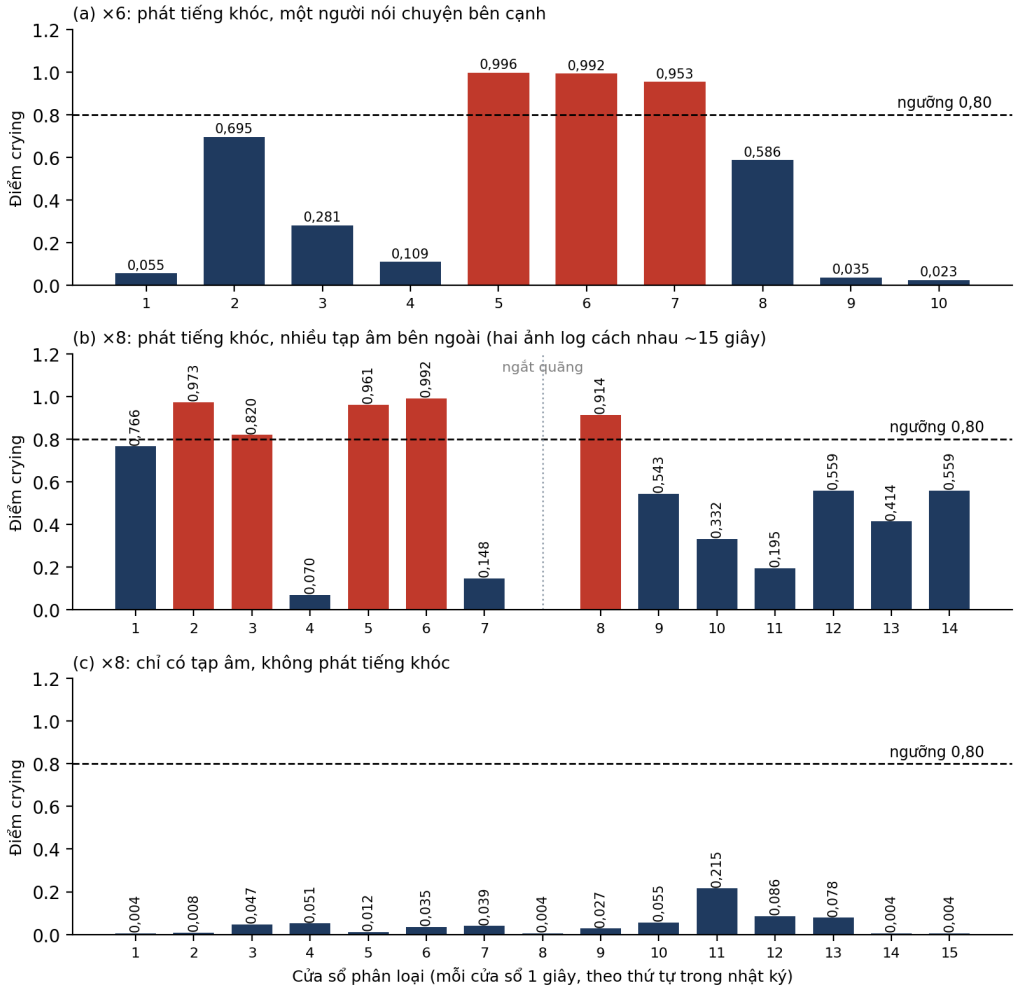
Ở ×1 tín hiệu chỉ chiếm 8,4% thang đo; ở ×8 đỉnh lên 28.000, gần mức bão hòa 32.767, và theo quan sát của nhóm hệ thống quá nhạy nên tiếng khóc dễ lẫn với tạp âm. Nhóm chọn MIC_GAIN = 6,0 (đỉnh 13.734, còn dư khoảng 2,4 lần trước khi bão hòa). Trong đoạn cuối nhật ký ×1 khi không còn

âm thanh, rms chỉ khoảng 38–61 (quy đổi khoảng −59 đến −55 dBFS), cho thấy nền nhiễu của micro rất thấp.

9.4. Kết quả nhận diện

Bảng 6. Số cửa sổ vượt ngưỡng 0,80 trong từng kịch bản (thư viện v1.0.3 xuất ngày 20/09/2026).

Hệ số	Kịch bản	Số cửa sổ có điểm	Số cửa sổ ≥ 0,80	Điểm crying cao nhất
×1	Khóc, nhiều tạp âm	18 (gồm đoạn cuối không có âm thanh)	1 (và 1 dòng thông báo bị cắt khỏi ảnh)	0,953
×6	Khóc, một người nói chuyện bên cạnh	10	3	0,996
×8	Khóc, nhiều tạp âm bên ngoài	14	5	0,992
×8	Chỉ có tạp âm	15	0	0,215



Hình 6. Điểm crying theo từng cửa sổ (cột đỏ: vượt ngưỡng 0,80).

- Không có báo giả trong 15 cửa sổ chỉ có tạp âm (điểm cao nhất 0,215).
- Độ nhạy theo cửa sổ còn khiêm tốn: 30% (3/10, ×6) và 36% (5/14, ×8) cửa sổ có tiếng khóc vượt ngưỡng; các cửa sổ còn lại từ 0,02 đến 0,77. Hai cửa sổ có biên độ gần tương đương có thể cho điểm rất khác nhau (ở ×6: peak 11.046, rms 1.977 cho 0,109; peak 9.288, rms 1.535 cho 0,996), nên điểm phụ thuộc hình dạng phổ hơn là độ lớn âm thanh.
- Chưa so sánh được các hệ số khuếch đại: mỗi kịch bản chỉ có 10–15 cửa sổ trong một đợt thử, và nhật ký ×1 cũng ghi nhận một lần phát hiện (0,953). Chênh lệch 30% và 36% nằm trong sai số lấy mẫu.
- Thời gian xử lý thực: DSP 45–49 ms và mạng 34 ms, tổng 79–83 ms, khoảng 8% chu kỳ 1 giây.

10. Hạn chế đã biết

- Bằng chứng phần cứng còn ít. Mỗi kịch bản chỉ là một đợt thử vài chục giây (10–15 cửa sổ); tiếng khóc được phát qua loa chứ không phải trẻ thật; khoảng cách và âm lượng chưa cố định.
- Số liệu Studio chưa chắc phản ánh môi trường thật. Tập kiểm định và kiểm thử cùng nguồn với dữ liệu huấn luyện; kết quả 92–94% chưa đảm bảo độ nhạy tương đương khi có tạp âm.
- Dữ liệu còn ít và hơi lệch. Tổng cộng 17 phút 7 giây, tiếng khóc chỉ khoảng 32%; một phần mẫu 1 giây có thể không phải do nhóm thu bằng INMP441; mẫu 10 giây dùng chung nhãn cho cả file.
- Lệch phiên bản giữa Studio và thiết bị. Thư viện xuất ngày 20/09/2026, còn dữ liệu và số liệu Studio mới hơn; chưa biết chính xác độ chính xác của mô hình đang chạy trên ESP32.
- Kết quả huấn luyện dao động giữa các lần chạy. Cùng cấu hình Add noise = Low nhưng accuracy là 91,9% và 89,8%, nên cần chạy nhiều lần trước khi kết luận.
- Thu và xử lý tuần tự, không chồng lấn. Âm thanh rơi vào lúc suy luận và gọi Firebase không được ghi lại; mỗi cửa sổ cần đủ 1 giây trước khi có kết quả.
- Phụ thuộc mức tín hiệu. Kết quả nhạy với MIC_GAIN và vị trí micro, vì hệ số khuếch đại dịch chuyển trực tiếp ảnh phổ (mục 5.2); chưa có cơ chế tự hiệu chỉnh.
- Chỉ có hai lớp. Mô hình chưa phân biệt tiếng khóc với tiếng cười; lớp tiếng cười được thử ở pipeline 3 lớp riêng và hay bị nhận nhầm thành tiếng khóc.

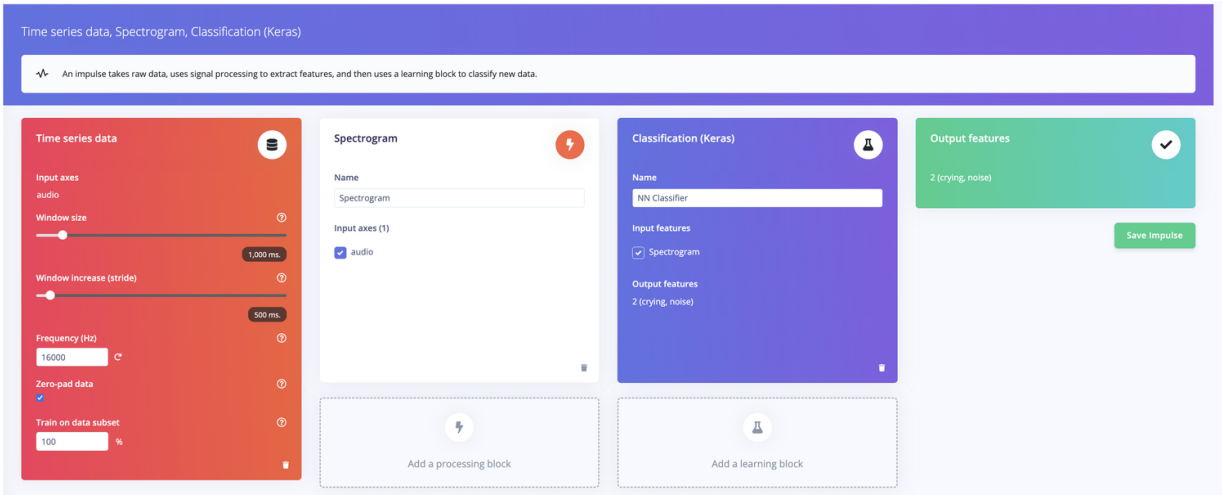
11. Hướng cải thiện theo thứ tự ưu tiên

Ưu tiên	Việc làm	Lý do	Công sức
1	Xuất lại thư viện từ lần huấn luyện cuối, nạp lên ESP32 rồi thử lại với cùng điều kiện như lần trước	Thư viện 20/09 cũ hơn dữ liệu và số liệu Studio; cần biết đúng mô hình đang chạy	Thấp
2	Thử nghiệm có kiểm soát: 20 đợt phát tiếng khóc và 20 đợt chỉ có tạp âm cho mỗi mô hình và mỗi MIC_GAIN, giữ cố định micro, âm lượng và đoạn âm thanh	Hiện chưa tính được tỉ lệ phát hiện và báo giả theo sự kiện	Thấp
3	Rà lại dữ liệu: xác nhận nguồn các mẫu 1 giây ngày 05/04/2022, cắt các mẫu 10 giây thành đoạn có nội dung đồng nhất, đếm lại số mẫu từng lớp	Tránh nhãn sai do đoạn yên tĩnh, và biết đúng tỉ lệ dữ liệu thu bằng INMP441	Thấp đến trung bình
4	Thu thêm dữ liệu bằng chính INMP441: tiếng khóc trộn với nền ồn thật (quán cà phê, xóm, tivi, quạt), nhất là thêm tiếng khóc	Tiếng khóc chỉ khoảng 32% dữ liệu; giảm lệch giữa dữ liệu huấn luyện và thiết bị thật	Trung bình
5	Chạy suy luận liên tục (4 lát × 250 ms) thay cho cửa sổ rời rạc 1 giây	Không bỏ sót âm thanh khi xử lý; kết quả mới sau mỗi 250 ms; thư viện đã có sẵn ví dụ	Trung bình
6	Thử augmentation từng tùy chọn riêng, mỗi cấu hình huấn luyện ít nhất 3 lần, rồi so sánh trên phần cứng	Add noise High hỏng; Low dao động giữa các lần chạy và chưa rõ lợi ích	Thấp đến trung bình
7	So sánh Spectrogram với MFE/MFCC trên cùng dữ liệu	Lựa chọn Spectrogram mới là lập luận thiết kế, chưa có số liệu	Thấp
8	Giảm số tham số của lớp cuối (thêm pooling hoặc lớp gộp theo thời gian) rồi đánh giá lại accuracy, loss và bộ nhớ	87% tham số nằm ở lớp cuối; mạng nhạy với vị trí trong cửa sổ	Trung bình
9	Mở rộng lên 3 lớp (thêm tiếng cười) sau khi bổ sung dữ liệu đa dạng	Lớp tiếng cười hiện hay bị nhầm thành tiếng khóc	Cao

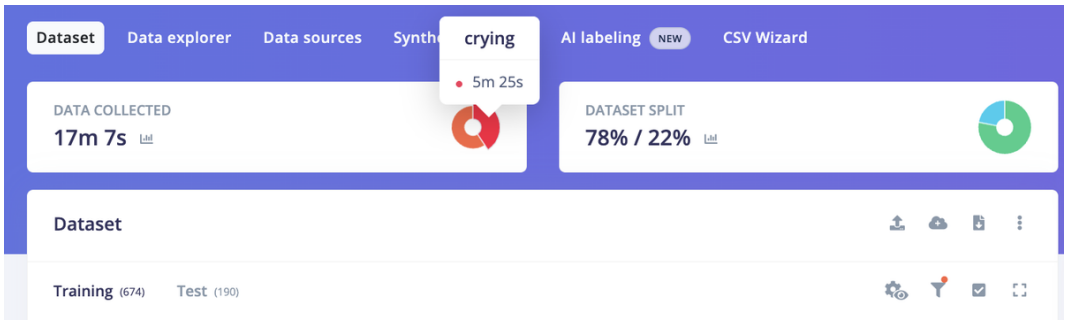
Phụ lục A. Cách tái lập và kiểm tra thư viện

1. Create impulse: Time series data (trục audio, 16.000 Hz, window size 1.000 ms, window increase 500 ms, bật zero-pad, Train on data subset 100%) → khối Spectrogram (frame length 0,02; frame stride 0,01; FFT length 128; noise floor −52 dB) → Classification (Keras) tên NN Classifier, đầu ra hai nhãn crying và noise.
2. NN settings: 100 chu kỳ, learning rate 0,008; kiến trúc Reshape (65 cột) → 1D conv/pool (8 filter, kernel 3) → Dropout 0,45 → Dense 256 → Dropout 0,45 → Flatten → Output 2 lớp.
3. Save & train, sau đó chạy Model testing trên tập kiểm thử.
4. Deployment: chọn Arduino library (không phải C++ library), bật EON Compiler, chọn Quantized (int8), rồi Build.
5. Cài thư viện: xóa hẳn thư mục thư viện cũ trong Documents/Arduino/libraries trước khi cài bản mới.
6. Kiểm tra trong Serial khi khởi động: tên project, ID 1117295, tần số 16.000 Hz, 16.000 mẫu, 2 lớp, NN input 6.435, và chỉ có 1 khối DSP.
7. Biên dịch firmware với Partition Scheme Huge APP (3 MB), vì firmware đầy đủ vượt phân vùng mặc định 1.310.720 byte.

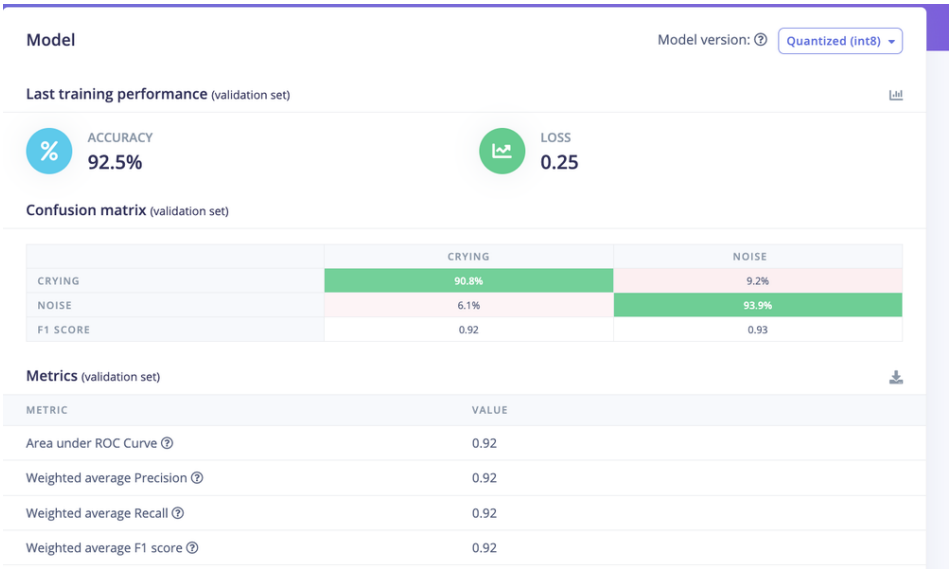
Phụ lục B. Ảnh chụp từ Edge Impulse Studio



Hình B1. Create impulse: cửa sổ 1.000 ms, bước 500 ms, 16.000 Hz, Spectrogram và Classification (Keras).



Hình B2. Tab Dataset: 17 phút 7 giây, tỉ lệ 78% / 22%, 674 mẫu huấn luyện và 190 mẫu kiểm thử; lớp crying 5 phút 25 giây.



Model — Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 92.5% — LOSS 0.25

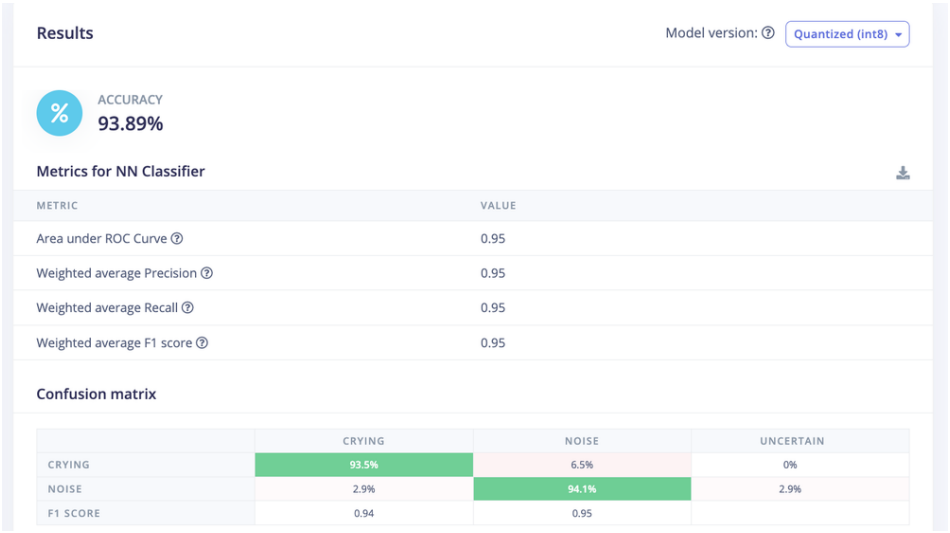
Confusion matrix (validation set)

	CRYING	NOISE
CRYING	90.8%	9.2%
NOISE	6.1%	93.9%
F1 SCORE	0.92	0.93

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	0.92
Weighted average Precision	0.92
Weighted average Recall	0.92
Weighted average F1 score	0.92

Hình B3. Training output (chưa bật tăng cường dữ liệu): accuracy 92,5%, loss 0,25.



Results — Model version: Quantized (int8)

ACCURACY 93.89%

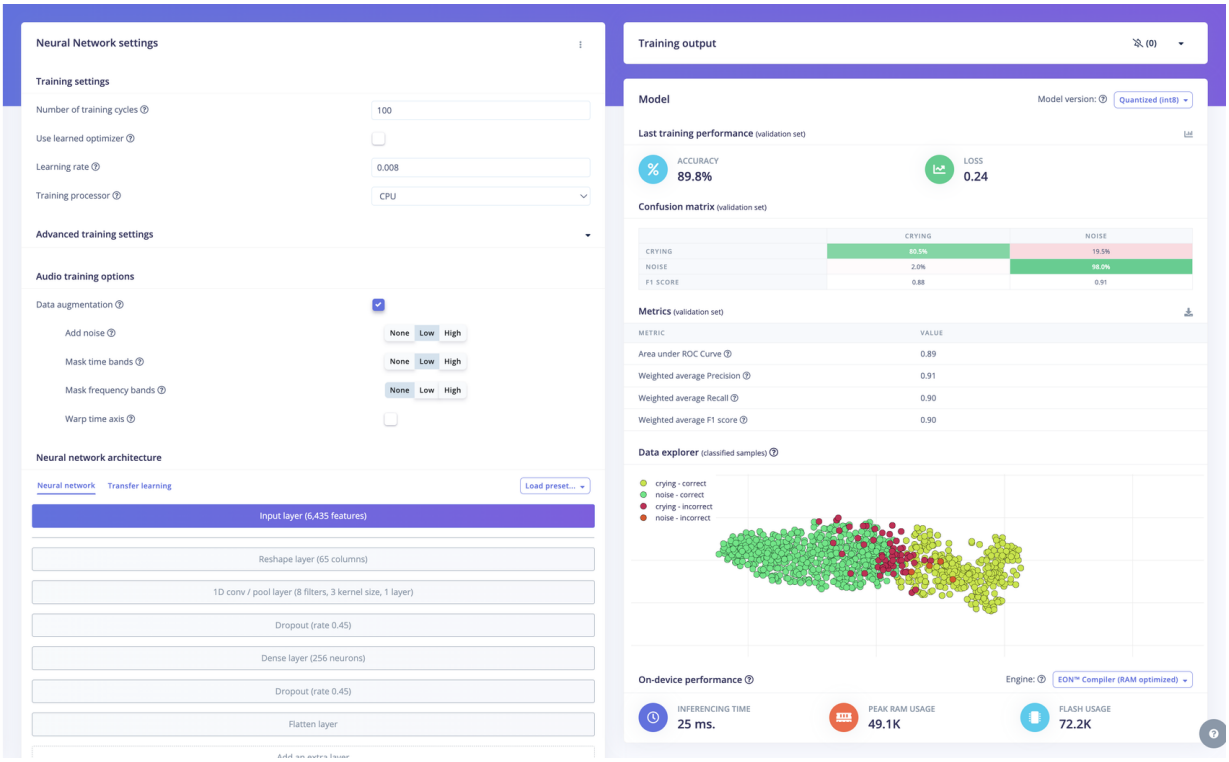
Metrics for NN Classifier

METRIC	VALUE
Area under ROC Curve	0.95
Weighted average Precision	0.95
Weighted average Recall	0.95
Weighted average F1 score	0.95

Confusion matrix

	CRYING	NOISE	UNCERTAIN
CRYING	93.5%	6.5%	0%
NOISE	2.9%	94.1%	2.9%
F1 SCORE	0.94	0.95	

Hình B4. Model testing: accuracy 93,89%.



Hình B5. Neural Network settings và Training output, bật Data augmentation với Add noise = Low: accuracy 89,8%, loss 0,24.

Phụ lục C. Nguồn của số liệu

Số liệu	Nguồn
Cửa sổ 16.000 mẫu, 16 kHz, 6.435 đặc trưng, 2 nhãn, ngưỡng 0,6, 4 lát, deploy version 3	model-parameters/model_metadata.h
Tham số khối Spectrogram, thứ tự nhãn, tên impulse	model-parameters/model_variables.h
Cấu trúc mạng, kích thước tensor, tham số, lượng tử hóa, tensor arena	tflite-model/tflite_learn_1117295_5_compiled.cpp
Thuật toán chia khung, FFT, chuẩn hóa dB	edge-impulse-sdk: classifier/ei_run_dsp.h, dsp/speechpy/feature.hpp, processing.hpp, dsp/numpy.hpp
Tham số huấn luyện, kiến trúc trên Studio	Ảnh chụp Neural Network settings
Accuracy, ma trận nhầm lẫn, ước tính RAM/flash/thời gian suy luận	Ảnh chụp Training output, Model testing và các lần thử Add noise
Thời gian DSP/NN, biên độ, điểm số theo cửa sổ	Nhật ký Serial Monitor (27/09/2026)
Số mẫu, thời lượng, tỉ lệ chia train/test, độ dài mẫu, cửa sổ và bước, zero-pad	Ảnh chụp tab Dataset và Create impulse
Nguồn dữ liệu 60% INMP441 / 40% công khai, các loại tạp âm	Nhóm cung cấp (chưa đối chiếu với Studio)
Dung lượng firmware, RAM tĩnh	Thông báo biên dịch Arduino IDE (bản firmware có Firebase)
MIC_GAIN, ngưỡng hysteresis, trường Firebase	Firmware smart_acoustic_node_ei v3 đến v5